DATA ENGINEERING & CLOUD · LESSON 12 OF 15

Containers, Kubernetes and CI/CD for Data Pipelines

Package a data pipeline so that it runs the same way on any machine. Write a Dockerfile, understand what Kubernetes adds for scheduling and scaling, and build a continuous integration and deployment pipeline that tests a dbt or Python project before it reaches production.

Gap-fill lesson · ✦ 2026 · Advanced · ⏱ 120 min

By the end of this lesson you can

- Explain images, containers and why they make pipelines portable
- Write a Dockerfile for a Python data job and run it with mounted data
- Describe Kubernetes objects and when they are needed for data work
- Design a CI/CD pipeline that runs tests and deploys a data project safely

Why containers

A pipeline that works on one laptop often fails on a server: a different Python version, a missing library, a path that does not exist. A **container** packages the code and its dependencies into a single unit that runs the same way everywhere. An **image** is the packaged recipe; a **container** is a running copy of it.

CASE STUDY **"It works on my machine"**

A data job needed a specific pandas version and a system library. Each engineer installed them differently, and the nightly server had a third setup. Once the job ran in a container built from one Dockerfile, every environment behaved the same, and the nightly failures stopped.

A Dockerfile for a data job

A **Dockerfile** lists the steps that build an image. The example below packages a Python job that reads a CSV and writes a summary. It is shown as code and **not run**, because it needs Docker

installed.

DOCKERFILE

```
# Dockerfile
FROM python:3.12-slim

WORKDIR /app

# Install dependencies first: this layer is cached until requirements.txt changes
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy the code last: code changes do not reinstall dependencies
COPY summarise_sales.py .

# Data is mounted at run time, not copied into the image
ENTRYPOINT ["python", "summarise_sales.py"]
```

Build and run it:

Terminal

```
docker build -t sales-summary:1.0 .
docker run --rm -v "$PWD/data:/data" sales-summary:1.0 --input /data/daily_sales.csv --
output /data/summary.csv
```

Key ideas:

- **Layer order** matters. Put the things that change least (dependencies) first, so rebuilds are fast.
- **Pin versions** in `requirements.txt` so that a rebuild next month does not change behaviour.
- **Do not copy secrets into the image.** Pass them at run time as environment variables or from a secrets store.
- **Mount data** rather than baking it in. The same image can then process today's file or last month's.

::::warn Images are not secret vaults Anything written into an image layer can be read by anyone who can pull the image, even if a later layer deletes it. Keep credentials out of the Dockerfile and out of the build context. ::::

What Kubernetes adds

Kubernetes runs containers across a cluster of machines. You describe the desired state (for example, "run three copies of this worker, restart it if it crashes, give it 2 GB of memory"), and Kubernetes keeps reality matching that description.

Kubernetes object	Purpose in data work
Pod	One or more containers that run together
Deployment	Keeps a long-running service (for example a streaming consumer) alive
Job / CronJob	Runs a batch task to completion, or on a schedule
ConfigMap and Secret	Configuration and credentials, kept out of the image
Horizontal autoscaler	Adds workers when a queue grows

YAML

```yaml
# A scheduled batch job (CronJob), shown as configuration; not run
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-sales-summary
spec:
  schedule: "0 2 * * *"           # 02:00 every day
  concurrencyPolicy: Forbid       # never start a second run while one is running
  jobTemplate:
    spec:
      backoffLimit: 3             # retry up to three times
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: summary
              image: registry.example.com/sales-summary:1.0
              envFrom:
                - secretRef:
                    name: warehouse-credentials
```

Kubernetes is powerful and has a learning cost. For a single nightly job, a managed scheduler (for example Airflow or a cloud scheduler) may be simpler. Use Kubernetes when you run many services or need fine control over scaling and placement.

CI/CD for data projects

Continuous integration (CI) runs the tests and checks on every change. **Continuous deployment (CD)** releases a tested version to an environment automatically or with an approval step. For data, the pipeline should also run the data tests, not only the code tests.

A sensible pipeline for a dbt or Python data project:

YAML

```yaml
# Illustrative CI workflow (GitHub Actions style); adapt to your platform
name: data-ci
on:
  pull_request:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.12"}
      - run: pip install -r requirements.txt
      - run: python -m pytest tests/            # unit tests for transformation code
      - run: dbt build --target ci              # models and their data tests in a CI
schema
  deploy-staging:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - run: dbt build --target staging
```

The `dbt build --target ci` step builds into a temporary schema, so the tests run on real data structures without touching production. Deploy to staging only after the tests pass, and promote to production with an approval.

::::tip Test data, not only code A passing unit test does not prove the table is right. Include data tests (uniqueness, not-null, reconciliation totals) in the CI run, and fail the pipeline when they fail. A change that breaks a metric should be caught before the merge. ::::

Release habits

- Tag each release with a version number; record which version produced each output table.
- Keep environments separate (CI, staging, production) with their own credentials.
- Make deployments reversible: keep the previous image and the previous dbt state, so a rollback is one command.
- Never deploy by editing production by hand; the pipeline is the only route.

INTERVIEW CORNER

How would you ship a change to a data pipeline safely?

Work on a branch, run unit and data tests in CI against a temporary schema, compare the output metrics with the previous version, deploy to staging, then promote to production with an approval and a recorded version. Keep the previous version available so that a rollback is quick.

::::key What to remember - Containers package code and dependencies so that a pipeline runs the same everywhere. - Layer Dockerfiles for fast rebuilds, pin versions, and keep secrets out of images. - Kubernetes runs and scales containers; use it when the operational need justifies it. - CI runs unit and data tests on every change; CD promotes tested versions with approval. - Version every release and keep rollbacks simple. ::::

Quick quiz

5 questions · tap an answer to check it

1. What is the difference between an image and a container?

- A They are the same thing
- B An image is the packaged recipe; a container is a running copy of it
- C A container is a file on disk only
- D An image runs without a host

Answer: B. The image is built once; containers are started from it.

2. Why put the dependency installation before copying the code in a Dockerfile?

- A It is required by Docker
- B So that the dependency layer is cached and rebuilds after code changes are fast
- C It makes the image smaller
- D It installs the code twice

Answer: B. Layer order determines what Docker can reuse.

3. Why should secrets not be baked into an image?

A They are too large

B Anyone who can pull the image can read them, even if a later layer deletes them

C Images cannot run with secrets

D Secrets are always public

Answer: B. Image layers are readable by anyone with access.

4. What does `concurrencyPolicy: Forbid` do in a CronJob?

A It stops the job from running at all

B It prevents a new run starting while the previous run is still active

C It doubles the retries

D It removes the schedule

Answer: B. It stops overlapping runs.

5. What makes a CI pipeline valuable for data projects?

A It only checks the code style

B It runs data tests on a temporary schema before a change reaches production

C It deploys without review

D It runs once a year

Answer: B. Data tests catch errors that code tests cannot see.

EXERCISE **Write a CI plan**

For a data project you know, write a CI and CD plan with five steps: the trigger, the code tests, the data tests and where they run, the staging deployment, and the production promotion with its approval. Name the one control that would have stopped the most serious past failure in that project.

Show solution (worked answer)

Hints: keep credentials in the CI system's secrets store. For the data tests, choose the three most important checks for the project. The control should be a test or an approval that the failure would have hit.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.